Chapter 6: Basic SQL

Exam Preparation Notes

Fundamentals of Database Systems, 7th Edition — Elmasri & Navathe

Exam Weightage: Low — Focus on key concepts, syntax, and practical query examples

1. Introduction to SQL

SQL (Structured Query Language) is the standard language for relational database management. Originally derived from a language called SQUARE, it was later renamed SEQUEL (Structured English Query Language) and eventually shortened to SQL. It is the primary reason for the commercial success of relational databases.

Key Points:

- SQL is an informal rendering of the relational data model with practical syntax.
- It covers: Data Definition (DDL), Data Manipulation (DML), Transaction Control, Security, Indexing, and more.
- SQL uses the terms Table (relation), Row (tuple), and Column (attribute).
- SQL is NOT purely set-based — it allows duplicate tuples (multiset/bag behavior), unlike the strict relational model.

NOTE: SQL originated from tuple relational calculus and was first described in IBM's SEQUEL TO SQUARE paper by Chamberlin and Boyce.

---

2. SQL Standards & Schema Concepts

2.1 SQL Standards Timeline

Standard	Key Feature
SQL-86 / SQL-1	Original standard
SQL-92 / SQL-2	Major revision, widely referenced
SQL-1999 / SQL-3	Current standard; core + extensions; OO features
SQL-2006	Added XML features (Ch.13)
SQL-2008	Added Object-Oriented features (Ch.12)

2.2 Schema and Catalog

- A Schema is identified by a name and includes an authorization identifier plus descriptors for each element (tables, constraints, views, domains).
- A Catalog is a named collection of schemas.
- A Cluster is a collection of catalogs.

```
-- Creating a schema
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

3. The CREATE TABLE Command

The CREATE TABLE command defines a new base relation. You provide the table name and specify each attribute with its data type and any constraints.

```
-- Basic syntax
CREATE TABLE table_name (
    attribute1  datatype  [constraint],
    attribute2  datatype  [constraint],
    ...
    [table_constraints]
);
```

- **Base tables**: actually created and stored as a file by the DBMS.
- **Virtual relations (Views)**: created with CREATE VIEW; no physical file.

NOTE: If FOREIGN KEY constraints cause circular reference errors (e.g., EMPLOYEE references DEPARTMENT and vice versa), the DBA can temporarily disable referential integrity enforcement during table creation.

4. Attribute Data Types

4.1 Numeric Types

Data Type	Description
INTEGER / INT	Whole numbers
SMALLINT	Smaller whole numbers
FLOAT / REAL	Floating-point numbers
DOUBLE PRECISION	Higher-precision floating-point
DECIMAL(p, s)	Exact numeric with p digits total, s after decimal

4.2 String Types

Data Type	Description
CHAR(n)	Fixed-length string of n characters
VARCHAR(n)	Variable-length string up to n characters
BIT(n)	Fixed-length bit string
BIT VARYING(n)	Variable-length bit string
BOOLEAN	TRUE, FALSE, or NULL

4.3 Date and Time Types

Data Type	Description
DATE	YYYY-MM-DD (Year, Month, Day)
TIME	HH:MM:SS
TIMESTAMP	DATE + TIME + fractional seconds
INTERVAL	Relative time value (used to increment/decrement dates)

4.4 Domain and User-Defined Types

- DOMAIN: A named data type to improve readability and make schema changes easier.

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

- TYPE: User-Defined Types (UDTs) are used for object-oriented applications.

```
CREATE TYPE SSN_TYPE AS CHAR(9) FINAL;
```

5. Specifying Constraints in SQL

5.1 Three Basic Constraint Types

Constraint Type	Rule
Key Constraint	Primary key values must be UNIQUE (no duplicates)
Entity Integrity	Primary key values must NOT be NULL
Referential Integrity	Foreign key value must exist as a primary key in the referenced table (or be NULL)

5.2 Attribute-Level Constraints

- NOT NULL: Prevents null values for an attribute.
- DEFAULT: Assigns a default value if none is provided.
- CHECK: Enforces a condition on attribute values.

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
Salary DECIMAL(10,2) DEFAULT 0.00;
```

5.3 PRIMARY KEY and UNIQUE

```
-- Inline (single attribute)
Dnumber INT PRIMARY KEY,

-- Table-level (single or composite)
PRIMARY KEY (Essn, Pno),

-- UNIQUE (candidate key / alternate key)
```

```
UNIQUE (Dname),
```

NOTE: PRIMARY KEY implicitly enforces NOT NULL. UNIQUE allows NULL values.

5.4 FOREIGN KEY and Referential Actions

By default, any operation that violates referential integrity is REJECTED. However, you can specify triggered actions:

Action	Effect
SET NULL	Sets the FK column to NULL when the referenced row is deleted/updated
CASCADE	Deletes/updates child rows automatically when parent is deleted/updated
SET DEFAULT	Sets the FK column to its default value
NO ACTION (default)	Rejects the operation

```
FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
    ON DELETE SET NULL
    ON UPDATE CASCADE,

FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
    ON DELETE SET DEFAULT
    ON UPDATE CASCADE,
```

NOTE: CASCADE is suitable for 'relationship' tables (e.g., WORKS_ON). SET NULL is suitable for optional references.

5.5 Named Constraints

Constraints can be named using the CONSTRAINT keyword — useful for later altering or dropping them:

```
CONSTRAINT EMPPK PRIMARY KEY (Ssn),
CONSTRAINT EMPSUPERFK
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
    ON DELETE SET NULL ON UPDATE CASCADE,
```

5.6 Tuple-Level CHECK Constraints

A CHECK clause at the end of CREATE TABLE applies to the entire tuple, not just one column:

```
CHECK (Dept_create_date <= Mgr_start_date);
```

6. CREATE TABLE — Full COMPANY Schema Example

The COMPANY database has 6 tables: EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT.

```
CREATE TABLE EMPLOYEE (
```

```
    Fname       VARCHAR(15)   NOT NULL,
    Minit       CHAR,
    Lname       VARCHAR(15)   NOT NULL,
    Ssn         CHAR(9)       NOT NULL,
    Bdate       DATE,
    Address     VARCHAR(30),
    Sex         CHAR,
    Salary      DECIMAL(10,2),
    Super_ssn   CHAR(9),
    Dno         INT           NOT NULL DEFAULT 1,
    PRIMARY KEY (Ssn),
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
        ON DELETE SET NULL ON UPDATE CASCADE,
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
        ON DELETE SET DEFAULT ON UPDATE CASCADE
);

CREATE TABLE DEPARTMENT (
    Dname           VARCHAR(15)   NOT NULL,
    Dnumber         INT           NOT NULL,
    Mgr_ssn         CHAR(9)       NOT NULL DEFAULT '888665555',
    Mgr_start_date  DATE,
    PRIMARY KEY (Dnumber),
    UNIQUE (Dname),
    FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
        ON DELETE SET DEFAULT ON UPDATE CASCADE
);

CREATE TABLE DEPT_LOCATIONS (
    Dnumber     INT           NOT NULL,
    Dlocation   VARCHAR(15)   NOT NULL,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
        ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE PROJECT (
    Pname       VARCHAR(15)   NOT NULL,
    Pnumber     INT           NOT NULL,
    Plocation   VARCHAR(15),
    Dnum        INT           NOT NULL,
    PRIMARY KEY (Pnumber),
    UNIQUE (Pname),
    FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber)
);

CREATE TABLE WORKS_ON (
    Essn    CHAR(9)       NOT NULL,
    Pno     INT           NOT NULL,
    Hours   DECIMAL(3,1)  NOT NULL,
    PRIMARY KEY (Essn, Pno),
```

```
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
    FOREIGN KEY (Pno)  REFERENCES PROJECT(Pnumber)
);

CREATE TABLE DEPENDENT (
    Essn            CHAR(9)       NOT NULL,
    Dependent_name  VARCHAR(15)   NOT NULL,
    Sex             CHAR,
    Bdate           DATE,
    Relationship    VARCHAR(8),
    PRIMARY KEY (Essn, Dependent_name),
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn)
);
```

7. Basic Retrieval Queries (SELECT)

7.1 SELECT-FROM-WHERE Structure

```
SELECT  <attribute list>
FROM    <table list>
[WHERE  <condition>]
[ORDER BY <attribute list>];
```

- **`SELECT`**: Specifies the attributes (columns) to retrieve — the projection.
- **`FROM`**: Lists the tables involved.
- **`WHERE`**: Boolean condition (selection predicate). Can include join conditions when multiple tables are used.

Logical comparison operators: =, <, <=, >, >=, <> (not equal)

7.2 Query Examples

Query 0 — Single table, specific row

Retrieve the birth date and address of the employee named John B. Smith.

```
SELECT  Bdate, Address
FROM    EMPLOYEE
WHERE   Fname='John' AND Minit='B' AND Lname='Smith';
```

Query 1 — Join of two tables

Retrieve the name and address of all employees who work for the Research department.

```
SELECT  Fname, Lname, Address
FROM    EMPLOYEE, DEPARTMENT
WHERE   Dname='Research' AND Dnumber=Dno;
```

Query 2 — Join of three tables

For every project in Stafford, list the project number, controlling department number, and the department manager's last name, address, and birth date.

```
SELECT  Pnumber, Dnum, Lname, Address, Bdate
FROM    PROJECT, DEPARTMENT, EMPLOYEE
WHERE   Dnum=Dnumber AND Mgr_ssn=Ssn
        AND Plocation='Stafford';
```

7.3 Ambiguous Attribute Names

When two tables have a column with the same name, qualify the column with the table name:

```
SELECT  Fname, EMPLOYEE.Name, Address
FROM    EMPLOYEE, DEPARTMENT
WHERE   DEPARTMENT.Name='Research'
        AND DEPARTMENT.Dnumber=EMPLOYEE.Dnumber;
```

7.4 Aliases and Tuple Variables

Use AS to create aliases for tables (essential for self-joins) or to rename attributes:

```
-- Query 8: Each employee with their supervisor's name
SELECT  E.Fname, E.Lname, S.Fname, S.Lname
FROM    EMPLOYEE AS E, EMPLOYEE AS S
WHERE   E.Super_ssn = S.Ssn;

-- Renaming attributes in alias
EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)
```

NOTE: The AS keyword can be omitted in most SQL implementations. Aliases are essential for self-joins (same table used twice).

7.5 Unspecified WHERE Clause and SELECT *

- Omitting WHERE results in a CROSS PRODUCT (Cartesian product) — all combinations of tuples from the listed tables.
- Use SELECT * to retrieve ALL columns of selected tuples.

```
-- Q9: All employee SSNs
SELECT Ssn FROM EMPLOYEE;

-- Q10: Cartesian product of EMPLOYEE and DEPARTMENT
SELECT Ssn, Dname FROM EMPLOYEE, DEPARTMENT;

-- Q1C: All columns of employees in department 5
SELECT * FROM EMPLOYEE WHERE Dno=5;
```

7.6 Eliminating Duplicates — DISTINCT

SQL keeps duplicates by default. Use DISTINCT to remove them:

```
-- Q11: All salaries (duplicates kept)
SELECT ALL Salary FROM EMPLOYEE;
```

```
-- Q11A: Unique salary values only
SELECT DISTINCT Salary FROM EMPLOYEE;
```

7.7 Set Operations: UNION, EXCEPT, INTERSECT

These operate on two queries that must be type-compatible (same number of attributes, compatible types):

Operation	Meaning	Multiset Version
UNION	All tuples from either query (no duplicates)	UNION ALL
EXCEPT	Tuples in first but not second	EXCEPT ALL
INTERSECT	Tuples in both queries	INTERSECT ALL

Query 4 — UNION Example

List all project numbers for projects involving an employee with last name Smith (as worker or as manager).

```
(SELECT DISTINCT Pnumber
 FROM    PROJECT, DEPARTMENT, EMPLOYEE
 WHERE   Dnum=Dnumber AND Mgr_ssn=Ssn AND Lname='Smith')
UNION
(SELECT DISTINCT Pnumber
 FROM    PROJECT, WORKS_ON, EMPLOYEE
 WHERE   Pnumber=Pno AND Essn=Ssn AND Lname='Smith');
```

7.8 Pattern Matching — LIKE

Used for string comparison with wildcards:

- % — Matches any sequence of zero or more characters
- _ (underscore) — Matches exactly one character

```
-- Employees living in Houston, TX
WHERE Address LIKE '%Houston,TX%';

-- SSN matching a specific pattern
WHERE Ssn LIKE '__1__8901';

-- Names starting with 'Al'
WHERE Fname LIKE 'Al%';
```

7.9 BETWEEN Operator

```
-- Employees in dept 5 with salary between 30000 and 40000
WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

7.10 Arithmetic in SELECT

You can use +, -, *, / in the SELECT clause:

```
-- Query 13: Projected salaries after 10% raise for ProductX workers
SELECT  E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM    EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE   E.Ssn=W.Essn AND W.Pno=P.Pnumber AND P.Pname='ProductX';
```

7.11 ORDER BY Clause

Sort the result set. Default is ascending (ASC); use DESC for descending:

```
SELECT  D.Dname, E.Lname, E.Fname, E.Salary
FROM    DEPARTMENT AS D, EMPLOYEE AS E
WHERE   D.Dnumber = E.Dno
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC;
```

NOTE: ORDER BY is placed at the END of the query. You can sort by multiple columns.

8. INSERT, DELETE, and UPDATE

8.1 INSERT

Adds one or more tuples to a table. Values must match the column order defined in CREATE TABLE.

```
-- U1: Insert a single tuple (all values provided in order)
INSERT INTO EMPLOYEE
VALUES ('Richard','K','Marini','653298653','1962-12-30',
        '98 Oak Forest, Katy, TX','M',37000,'653298653',4);

-- Insert with explicit column list (order can differ)
INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno)
VALUES ('John', 'Doe', '111222333', 5);

-- U3B: Insert multiple tuples from a query result
INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours_per_week)
    SELECT E.Lname, P.Pname, W.Hours
    FROM   PROJECT P, WORKS_ON W, EMPLOYEE E
    WHERE  P.Pnumber=W.Pno AND W.Essn=E.Ssn;
```

Bulk Loading with LIKE and DATA

```
-- Create and load a new table like EMPLOYEE, only dept 5 records
CREATE TABLE D5EMPS LIKE EMPLOYEE
    (SELECT E.*
     FROM   EMPLOYEE AS E
     WHERE  E.Dno=5)
WITH DATA;
```

8.2 DELETE

Removes rows from a table. The WHERE clause determines which rows are affected.

- No WHERE clause = ALL rows deleted (table becomes empty).
- Only deletes from ONE table at a time (unless CASCADE is triggered).
- Referential integrity is enforced — you cannot delete a row referenced by a FK unless CASCADE or SET NULL is defined.

```
-- U4A: Delete employees with last name Brown
DELETE FROM EMPLOYEE WHERE Lname='Brown';

-- U4B: Delete a specific employee by SSN
DELETE FROM EMPLOYEE WHERE Ssn='123456789';

-- U4C: Delete all employees in department 5
DELETE FROM EMPLOYEE WHERE Dno=5;

-- U4D: Delete ALL employee records (table still exists, just empty)
DELETE FROM EMPLOYEE;
```

8.3 UPDATE

Modifies attribute values in existing rows. Uses SET to specify new values and WHERE to select which rows.

```
-- U5: Change location and department of project 10
UPDATE PROJECT
SET    Plocation='Bellaire', Dnum=5
WHERE  Pnumber=10;

-- U6: Give all Research department employees a 10% raise
UPDATE EMPLOYEE
SET    Salary = Salary * 1.1
WHERE  Dno IN (SELECT Dnumber
               FROM   DEPARTMENT
               WHERE  Dname='Research');
```

NOTE: In the SET clause, the attribute on the RIGHT of = refers to the OLD value before modification. The attribute on the LEFT refers to the new value being stored.

9. Quick Reference Summary

9.1 Key SQL Commands at a Glance

Command	Purpose
CREATE SCHEMA	Define a new schema
CREATE TABLE	Define a new table with columns and constraints
SELECT ... FROM ... WHERE	Retrieve data from tables
INSERT INTO	Add new rows to a table
DELETE FROM	Remove rows from a table

Command	Purpose
UPDATE ... SET	Modify existing rows
ORDER BY	Sort the result set (ASC / DESC)
DISTINCT	Remove duplicate rows from results
LIKE	String pattern matching (% and _)
BETWEEN	Range condition (inclusive)
UNION / EXCEPT / INTERSECT	Set operations on query results

9.2 Constraint Summary

Keyword	Meaning
PRIMARY KEY	Unique identifier; NOT NULL enforced automatically
UNIQUE	Alternate key; NULLs may be allowed
NOT NULL	Column must have a value
DEFAULT value	Value used when none is specified on INSERT
CHECK (condition)	Validates column or row values
FOREIGN KEY ... REFERENCES	Enforces referential integrity
ON DELETE / ON UPDATE	Triggered referential action (SET NULL, CASCADE, SET DEFAULT)
CONSTRAINT name	Gives a constraint a name for later modification

9.3 Common Exam Traps

- SQL allows duplicate rows — you must use DISTINCT explicitly to eliminate them.
- Missing WHERE in DELETE removes ALL rows from the table (table still exists).
- Missing WHERE in SELECT with multiple tables produces a Cartesian product.
- PRIMARY KEY automatically enforces NOT NULL; UNIQUE does not.
- CASCADE on DELETE propagates deletes to child tables.
- BETWEEN is inclusive on both ends: BETWEEN 30000 AND 40000 includes 30000 and 40000.
- The % wildcard in LIKE matches zero or more characters; _ matches exactly one.
- Aliases are required for self-joins — the same table appears twice with different names.
- Foreign key circular references must be handled by the DBA (e.g., by deferring constraint checks).

End of Chapter 6 Notes

Good luck on your exam!